



Unione di Cervelli per l'apprendimento di software Open Source

Sistemi e Reti: 3CIIN - 2021/2022

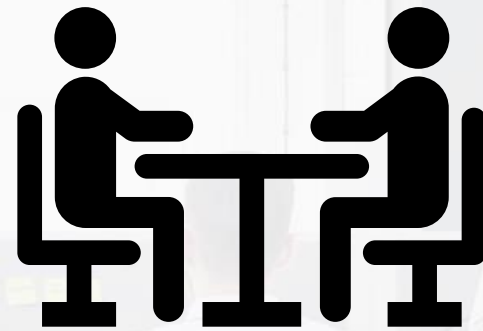
Prof. Benzi Luca – Prof. Antonella Zancanaro

Obiettivi dell'attività didattica

- Maturare conoscenze di software Open Source
- Sviluppare apprendimento autonomo guidato
- Incrementare abilità di collaborazione in team
- Utilizzare il web in modo costruttivo
- Trasmettere ai compagni le conoscenze acquisite in modo efficace

Regole

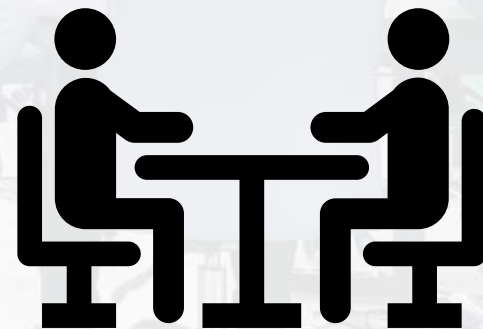
- La classe si suddivide in 4 sale
- Ogni sala nomina un caposala
- Ad ogni sala sarà assegnata una tecnologia Open Source che deve essere:
 - Studiata
 - Approfondita
 - Saputa applicare in pratica
 - Saputa spiegare e presentare



Sala Jobs



Sala Gates



Sala Bezos



Sala Musk

Criteri di valutazione

- Ogni gruppo prepara una presentazione di 40 - 50 minuti così strutturata:
 - Introduzione della tecnologia: 0 - 10 min
 - Dettaglio delle funzionalità tecniche: 30 - 40 min in cui si alternano:
 - Spiegazione concetto
 - Dimostrazione pratica
 - Interazione con il pubblico
 - Dimostrazione del progetto realizzato: 10 minuti

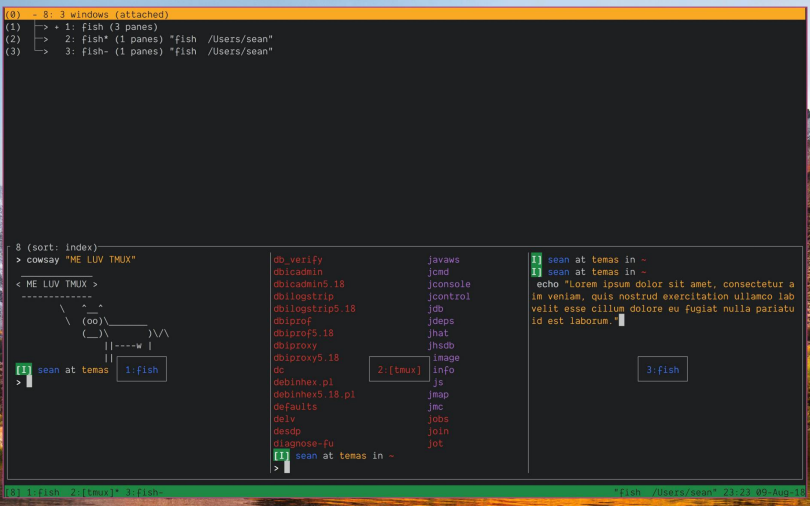


Alla presentazione migliore viene regalato 1 punto omaggio





Quattro progetti fantastici



main 3 branches 12 tags

Go to file Add file Code

zerowidth Merge pull request #119 from ptrykov/unused-attribute ✓ 04dd1ba on Jun 2 165 commits

doc	Update changelog and version for 1.3.0 release	17 months ago
lib	Unused attribute removed	3 months ago
script	End of support for version 2.2 or lower And Add newer version of ru...	2 years ago
test	Fix begin/rescue syntax for older rubies	11 months ago
.gitignore	Added Coveralls for code coverage tracking	5 years ago
.travis.yml	End of support for version 2.2 or lower And Add newer version of ru...	2 years ago
CONTRIBUTING.md	Fix typo in CONTRIBUTING.md	3 years ago
Gemfile		6 years ago
LICENSE.txt		6 years ago
README.md	Add paleontologist to alternatives	3 months ago
scientist.gemspec	End of support for version 2.2 or lower And Add newer version of ru...	2 years ago

README.md

Scientist!

A Ruby library for carefully refactoring critical paths. build passing coverage 99%

4

CONTENTS

4 Using Math 25

4.1 Math Mode 25

4.2 Navigating an Equation 26

4.3 Exponents and Indices 27

4.4 The Math toolbar 27

4.4.1 Greek and symbols 28

4.4.2 Roots, decorations, and delimiters 28

4.4.3 Fractions 29

4.4.4 Functions: lim, log-sin and others 29

4.4.5 Matrices 29

4.4.6 Display mode 30

4.5 More Math Stuff 31

5 Miscellaneous 33

5.1 Other major L^AT_EX Features 33

5.2 L^AT_EX for L^AT_EX Users 34

5.2.1 T_EX Mode 34

5.2.2 Importing L^AT_EX Documents to L^AT_EX 35

5.2.3 Converting L^AT_EX Documents to L^AT_EX 36

5.2.4 L^AT_EX Preamble 36

5.2.5 Bib_L 37

5.3 Errors! 37

Chapter 1

Introduction

1.1 Welcome to L^AT_EX!

This manual is designed for all of you who have never heard of L^AT_EX, or do not know it very well. Now, do not panic – you will not need to learn L^AT_EX to use L^AT_EX. That is, after all, the whole point of L^AT_EX: to provide an almost-WYSIWYG interface to L^AT_EX. There are some things you will need to learn, however, in order to use L^AT_EX effectively.

Some of you probably found your way to this document because you tried to put two spaces after a “.” or tried to put three blank lines between paragraphs. You found out you could not and, in fact, you will find out that most of the little tricks you are accustomed to use in word processors will not work in L^AT_EX. That is because most word processors you have used before allow you manually to enter all spacings, font changes, and so on. So you end up not only writing a document but typesetting it, too. L^AT_EX does the typesetting for you, in a consistent fashion, letting you focus on the important things, like the content of your writing.

So read on to learn more about L^AT_EX. Reading this tutorial is definitely worth the time.

1.2 What the Tutorial is and what it is not

Before we get started with this section, you need to know that this Tutorial uses the notation outlined in the *Introduction* manual. If you came to

5

#include <coroutine>
#include <iostream>
#include <stdexcept>
#include <thread>

auto switch_to_new_thread(std::jthread& out)
{
 struct available
 {
 std::jthread& p_out;

 bool await_ready() { return false; }

 void await_suspend(std::coroutine_handle<H> h)
 {
 std::jthread& out = *p_out;
 if (out.joinable())
 throw std::runtime_error("Output thread parameter not empty");
 out = std::jthread(h) { h.resume(); };
 // Potential undefined behavior: accessing potentially destroyed *this
 // std::cout << "New thread ID: " << p_out->get_id() << "\n";
 std::cout << "New thread ID: " << out.get_id() << "\n";
 // This is OK
 }

 void await_resume() {}
 };
 return available{out};
}

struct promise_type
{
 task get_return_object() { return {}; }
 std::suspend_never initial_suspend() { return {}; }
 std::suspend_never final_suspend() { return {}; }
 void return_void() {}
 void unhandled_exception() {}
};

codes/cpp/coroutines/coroutine.cpp 35 64 0

#include <coroutine>
#include <iostream>
#include <stdexcept>
#include <thread>

auto switch_to_new_thread(std::jthread& out)
{
 struct available
 {
 std::jthread& p_out;

 bool await_ready() { return false; }

 void await_suspend(std::coroutine_handle<H> h)
 {
 std::jthread& out = *p_out;
 if (out.joinable())
 throw std::runtime_error("Output thread parameter not empty");
 out = std::jthread(h) { h.resume(); };
 // Potential undefined behavior: accessing potentially destroyed *this
 // std::cout << "New thread ID: " << p_out->get_id() << "\n";
 std::cout << "New thread ID: " << out.get_id() << "\n";
 // This is OK
 }

 void await_resume() {}
 };
 return available{out};
}

struct task
{
 struct promise_type
 {
 task get_return_object() { return {}; }
 std::suspend_never initial_suspend() { return {}; }
 std::suspend_never final_suspend() { return {}; }
 void return_void() {}
 void unhandled_exception() {}
 };
};

codes/cpp/coroutines/coroutine.cpp 32 64 0

```
(0) - 8: 3 windows (attached)
(1)  |> + 1: fish (3 panes)
(2)  |> 2: fish* (1 panes) "fish /Users/sean"
(3)  |> 3: fish- (1 panes) "fish /Users/sean"

8 (sort: index)
> cowsay "ME LUV TMUX"
< ME LUV TMUX >
-----
      \   ^__^
       (oo)\_______
          (__)\       )\/\
              ||----w |
              ||

[I] sean at temas 1:fish
>

db_verify      javaws
dbicadmin      jcmd
dbicadmin5.18  jconsole
dbilogstrip    jcontrol
dbilogstrip5.18 jdb
dbiprof        jdeps
dbiprof5.18    jhat
dbiproxy       jhsdb
dbiproxy5.18   image
dc             2:[tmux] info
debinhex.pl    js
debinhex5.18.pl jmap
defaults       jmc
delv           jobs
desdp          join
diagnose-fu    jot
[I] sean at temas in ~
>

[I] sean at temas in ~
[I] sean at temas in ~
echo "Lorem ipsum dolor sit amet, consectetur a
im veniam, quis nostrud exercitation ullamco lab
velit esse cillum dolore eu fugiat nulla pariatu
id est laborum."

[8] 1:fish 2:[tmux]* 3:fish- "fish /Users/sean" 23:23 09-Aug-18
```

tMux: lavorare in multitasking con il terminale

tMux: lavorare in multitasking con il terminale

- Installare tMux
- Capire come attivare più terminali
- Acquisire dimestichezza con i comandi di tMux
- Creare un piano di lavoro che permette in contemporanea di:
 - Navigare tra file e cartelle
 - Editare un file
 - Navigare sul web da terminale
- Trasmettere ai compagni le conoscenze acquisite in modo efficace

4	CONTENTS	
4	Using Math	25
4.1	Math Mode	25
4.2	Navigating an Equation	26
4.3	Exponents and Indices	27
4.4	The Math toolbar	27
4.4.1	Greek and symbols	28
4.4.2	Roots, decorations, and delimiters	28
4.4.3	Fractions	29
4.4.4	Functions: lim, log, sin and others	29
4.4.5	Matrices	29
4.4.6	Display mode	30
4.5	More Math Stuff	31
5	Miscellaneous	33
5.1	Other major LyX Features	33
5.2	LyX for L ^A T _E X Users	34
5.2.1	T _E X Mode	34
5.2.2	Importing L ^A T _E X Documents — tex2lyx	35
5.2.3	Converting LyX Documents to L ^A T _E X	36
5.2.4	L ^A T _E X Preamble	36
5.2.5	BibT _E X	37
5.3	Errors!	37

Chapter 1

Introduction

1.1 Welcome to LyX!

This manual is designed for all of you who have never heard of L^AT_EX, or do not know it very well. Now, do not panic – you will not need to learn L^AT_EX to use LyX. That is, after all, the whole point of LyX: to provide an almost-WYSIWYG interface to L^AT_EX. There are some things you will need to learn, however, in order to use LyX effectively.

Some of you probably found your way to this document because you tried to put two spaces after a “.” or tried to put three blank lines between paragraphs. You found out you could not and, in fact, you will find out that most of the little tricks you are accustomed to use in word processors will not work in LyX. That is because most word processors you have used before allow you manually to enter all spacings, font changes, and so on. So you end up not only writing a document but typesetting it, too. LyX does the typesetting for you, in a consistent fashion, letting you focus on the important things, like the content of your writing.

So read on to learn more about LyX. Reading this tutorial is definitely worth the time.

1.2 What the Tutorial *is* and what it *is not*

Before we get started with this section, you need to know that this *Tutorial* uses the notation outlined in the *Introduction* manual. If you came to

Lyx: scrivere favolosi documenti scientifici

Lyx: scrivere favolosi documenti scientifici

- Installare Lyx
- Apprendere il significato di "What you see is what you get" – "What you see is not what you get"
- Apprendere le basi del linguaggio Latex e scrivere un documento scientifico su retta e piano cartesiano inserendo:
 - Sezioni
 - Paragrafi
 - Formule
 - Immagini
 - Sommario
- Trasmettere ai compagni le conoscenze acquisite in modo efficace

```

#include <coroutine>
#include <iostream>
#include <stdexcept>
#include <thread>

auto switch_to_new_thread(std::jthread& out)
{
    struct awaitable
    {
        std::jthread* p_out;

        bool await_ready() { return false; }

        void await_suspend(std::coroutine_handle<> h)
        {
            std::jthread& out = *p_out;
            if (out.joinable())
                throw std::runtime_error("Output jthread parameter not empty");
            out = std::jthread([h] { h.resume(); });
            // Potential undefined behavior: accessing potentially destroyed *this
            // std::cout << "New thread ID: " << p_out->get_id() << '\n';
            std::cout << "New thread ID: " << out.get_id()
                      << '\n'; // this is OK
        }

        void await_resume() {}
    };

    return awaitable{&out};
}

struct task
{
    struct promise_type
    {
        task get_return_object() { return {}; }
        std::suspend_never initial_suspend() { return {}; }
        std::suspend_never final_suspend() { return {}; }
        void return_void() {}
        void unhandled_exception() {}
    };
};

```

codes/cpp/coroutines/coroutine.cpp

35/64 : 0

```

#include <coroutine>
#include <iostream>
#include <stdexcept>
#include <thread>

auto switch_to_new_thread(std::jthread& out)
{
    struct awaitable
    {
        std::jthread* p_out;

        bool await_ready() { return false; }

        void await_suspend(std::coroutine_handle<> h)
        {
            std::jthread& out = *p_out;
            if (out.joinable())
                throw std::runtime_error("Output jthread parameter not empty");
            out = std::jthread([h] { h.resume(); });
            // Potential undefined behavior: accessing potentially destroyed *this
            // std::cout << "New thread ID: " << p_out->get_id() << '\n';
            std::cout << "New thread ID: " << out.get_id()
                      << '\n'; // this is OK
        }

        void await_resume() {}
    };

    return awaitable{&out};
}

struct task
{
    struct promise_type
    {
        task get_return_object() { return {}; }
        std::suspend_never initial_suspend() { return {}; }
        std::suspend_never final_suspend() { return {}; }
        void return_void() {}
        void unhandled_exception() {}
    };
};

```

codes/cpp/coroutines/coroutine.cpp

32/64 : 0

Vi – Vim: editor di testo o IDE?

Vi – Vim: editor di testo o IDE?

- Verificare se Vi è installato
- Apprendere i comandi principali e sviluppare dimestichezza
- Apprendere il meccanismo di funzionamento dei plugin
- Installare un plugin per "Syntax Highlighting" per il linguaggio C
- Sviluppare un programma C di base facendo uso dei costrutti visti nella materia Informatica e TDP.
- Trasmettere ai compagni le conoscenze acquisite in modo efficace

main 3 branches 12 tags

Go to file Add file Code

zerowidth Merge pull request #119 from ptrykov/ptrykov/unused-attribute 04dd1ba on Jun 2 165 commits

doc	Update changelog and version for 1.3.0 release	17 months ago
lib	Unused attribute removed	3 months ago
script	End of support for version 2.2 or lower And Add newer version of ru...	2 years ago
test	Fix begin/rescue syntax for older rubies	11 months ago
.gitignore	Added Coveralls for code coverage tracking	5 years ago
.travis.yml	End of support for version 2.2 or lower And Add newer version of ru...	2 years ago
CONTRIBUTING.md	Fix typo in CONTRIBUTING.md	3 years ago
Gemfile		6 years ago
LICENSE.txt		6 years ago
README.md	Add paleontologist to alternatives	3 months ago
scientist.gemspec	End of support for version 2.2 or lower And Add newer version of ru...	2 years ago

README.md

Scientist!

A Ruby library for carefully refactoring critical paths. build passing coverage 99%

Readme file: capire a colpo d'occhio lo scopo di un programma

Readme file: capire a colpo d'occhio lo scopo di un programma

- Capire cosa è un file "readme" e a che cosa serve
- Apprendere la sintassi del linguaggio di markdown
- Creare un file readme che descrive:
 - il programma C più complesso che avete sviluppato
 - come compilare un programma C in ambiente linux con i relativi comandi
- Trasmettere ai compagni le conoscenze acquisite in modo efficace